CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

PROJECT / PROBLEM BASED LEARNING (PBL)

A CASE STUDY ON

SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC)

Applied to an Online Library Management System



Submitted by	**MIMANSA**
UID	**25BCD10242**
Project type	**Case Study (Problem / Project Based Learning)**
Topic	**Software Development Life Cycle (SDLC)**
Month / Year	**November 2026**

INDEX

1. Abstract **3**
2. Introduction **3**
3. Objectives of the Case Study **3**
4. Case Background **4**
5. Problem Statement **4**
6. Scope and Proposed Solution **4**
7. Overview of the SDLC **5**
8. SDLC Models **6**
8.1 Waterfall Model 6
8.2 Agile (Scrum) Model 6
8.3 Spiral Model 7
8.4 Comparison of Models 7
8.5 Model Selected for the Case 7
9. Applying the SDLC to the Case **8**
9.1 Phase 1: Planning 8
9.2 Phase 2: Requirement Analysis 8
9.3 Phase 3: System Design (Flowchart, Use Case, DFDs) 9
9.4 Phase 4: Implementation 11
9.5 Phase 5: Testing 11
9.6 Phase 6: Deployment and Maintenance 11
10. Findings and Analysis **12**
11. Challenges and Lessons Learned **12**
12. Conclusion **12**
References **12**

List of Figures

Fig. 1 Phases of the SDLC 5
Fig. 2 Waterfall model 6
Fig. 3 Agile (Scrum) cycle 6
Fig. 4 Spiral model 7
Fig. 5 Project schedule (Gantt chart) 8
Fig. 6 Flowchart of the development process 9
Fig. 7 Use case diagram 9
Fig. 8 Level 0 DFD (context diagram) 10
Fig. 9 Level 1 DFD 10

1 Abstract

Every piece of software, from a small mobile app to a large banking system, goes through a series of stages from the first idea to the day it is retired. The Software Development Life Cycle (SDLC) is the structured framework that defines these stages. This case study examines the SDLC in a practical setting: the design and development of an Online Library Management System (OLMS) for a college library that currently runs on paper registers.

The study explains each SDLC phase, compares the Waterfall, Agile and Spiral models, selects the most suitable model for the case, and then walks through every phase with supporting artefacts such as a Software Requirement Specification (SRS), a process flowchart, a use case diagram, Data Flow Diagrams (DFDs), a project schedule and a test plan. The study concludes that a disciplined SDLC reduces cost, controls risk and improves the quality of the final product.

2 Introduction

Software engineering is the application of a systematic, disciplined and measurable approach to building software. Without such an approach, projects commonly run over budget, miss deadlines or fail to meet user expectations. The SDLC provides this approach by dividing the work into clear phases, each with defined inputs, activities and outputs.

The phases of the SDLC are Planning, Requirement Analysis, Design, Implementation, Testing, and Deployment & Maintenance. Different development models arrange these phases in different ways. Some run them one after the other, some repeat them in short cycles, and some add a risk check at every round.

> **In simple words:** The SDLC is like building a house. First you decide what you can afford (planning), then you list what rooms you need (requirements), then an architect draws the plan (design), builders construct it (coding), an inspector checks it (testing), the family moves in (deployment), and repairs are done over the years (maintenance).

3 Objectives of the Case Study

The objectives of this case study are:

- To understand the meaning, importance and phases of the Software Development Life Cycle.
- To compare the Waterfall, Agile and Spiral models and identify where each one fits best.
- To apply the SDLC step by step to a real-world style problem, the Online Library Management System.
- To prepare standard design artefacts: flowchart, use case diagram and Data Flow Diagrams.
- To draw findings and lessons that can be reused in future software projects.

4 Case Background

The case is based on a typical college library serving about 3,000 students and 150 faculty members. The library holds roughly 20,000 books and journals. The case is a representative, hypothetical scenario created for academic study, and its figures are illustrative.

Item	Description
Organisation	A typical college library (hypothetical case)
Users	Students, faculty, librarian and system administrator
Current system	Manual paper registers and card catalogue
Collection size	About 20,000 books and journals
Proposed system	Web-based Online Library Management System (OLMS)
Project team	1 project manager, 2 developers, 1 tester, 1 designer
Estimated duration	18 weeks

5 Problem Statement

The existing manual process creates several everyday difficulties for the library staff and its users:

- **Slow searching:** finding whether a book is available means walking to the shelf or flipping through catalogue cards.
- **Errors in records:** handwritten entries of issue and return dates are often incorrect or illegible.
- **Fine calculation mistakes:** late fees are calculated by hand, which causes disputes.
- **No reports:** the librarian cannot quickly see the most borrowed books or the list of defaulters.
- **Lost data:** paper registers can be damaged, lost or misplaced, and cannot be backed up.

Problem in one line: the library needs a fast, accurate and reliable computerised system to manage books, members and transactions.

6 Scope and Proposed Solution

The Online Library Management System will replace the paper registers. Its scope is limited to the following areas so that the project stays achievable within the planned time and budget.

In scope	Out of scope
Member registration and login	Online payment gateway
Book search by title, author or subject	E-book reading and downloads
Issue, return and renewal of books	Mobile application (planned for a later version)
Automatic fine calculation	Integration with other university software
Reports for the librarian and administrator	Barcode hardware procurement

7 Overview of the SDLC

Definition. The Software Development Life Cycle is a process that describes how software is planned, created, tested, delivered and maintained. It gives the team a common roadmap and gives the customer a way to track progress.

Why it matters. Following the SDLC helps to: (a) estimate cost and time realistically, (b) catch mistakes early when they are cheap to fix, (c) keep the whole team aligned, (d) produce documents that make later maintenance easier, and (e) deliver software that matches what the user actually needs.



Fig. 1: Phases of the Software Development Life Cycle

Phase	What happens	Output
1. Planning	Define the goal, check feasibility, estimate cost, time and staff.	Project plan, feasibility report
2. Requirement Analysis	Talk to users and collect what the system must do.	SRS document
3. Design	Decide how the system will work: screens, database, diagrams.	DFDs, UML, database design
4. Implementation	Developers write the code following the design.	Working software modules
5. Testing	Check the software for bugs and confirm it meets requirements.	Test cases, bug reports
6. Deployment & Maintenance	Release to users, then fix errors and add improvements.	Live system, updates

8 SDLC Models

A model is a particular way of arranging the SDLC phases. The three most widely studied models are described below.

8.1 Waterfall Model

The Waterfall model is the oldest and simplest model. The team completes one phase fully, signs it off, and only then moves to the next. Like water flowing down steps, it normally does not go back up.

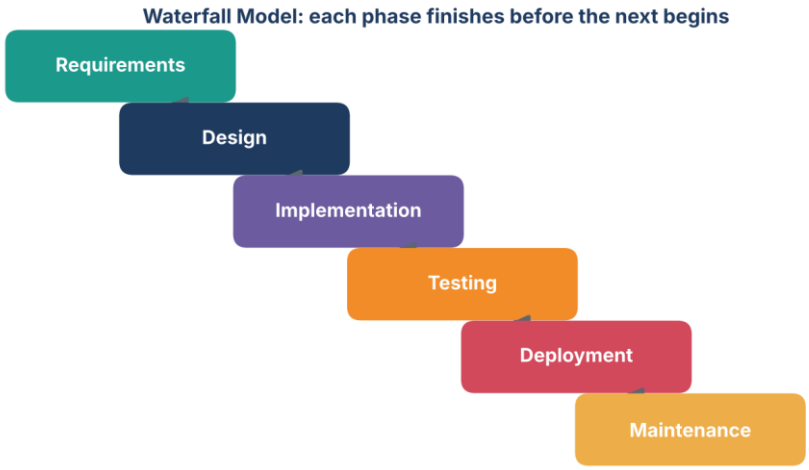


Fig. 2: Waterfall model

Advantages: easy to understand and manage; clear documents at every stage; works well when requirements are fixed. **Disadvantages:** changes are costly once a phase is closed; users see the product only at the end.

8.2 Agile (Scrum) Model

Agile builds the software in short cycles called sprints, usually one to four weeks long. At the end of each sprint the team delivers a small working increment and collects feedback. Scrum is the most popular Agile framework.

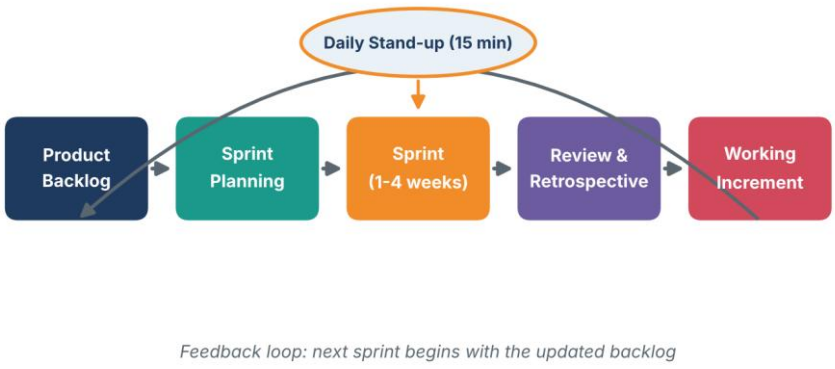


Fig. 3: Agile (Scrum) cycle

Advantages: welcomes changing requirements; frequent working releases; strong customer involvement. **Disadvantages:** needs an experienced and communicative team; less formal documentation.

8.3 Spiral Model

The Spiral model combines iterative development with a strong focus on risk. Each loop of the spiral passes through four steps: set objectives, identify and resolve risks, develop and test, and plan the next loop. The project grows outward with each loop, like a spiral.

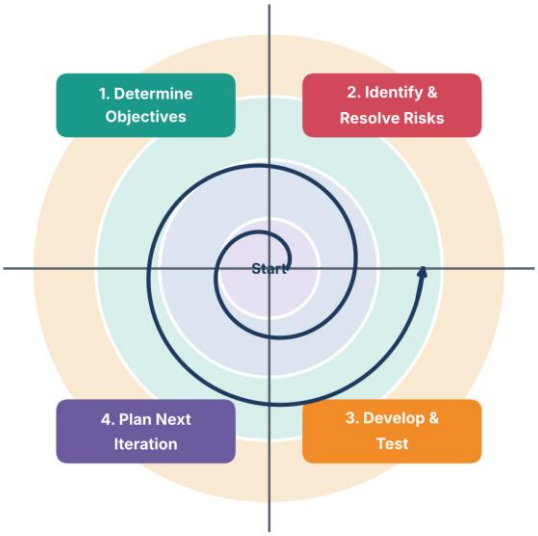


Fig. 4: Spiral model

Advantages: best for large, risky projects; risks are handled early. **Disadvantages:** expensive; needs skilled risk analysts; not suitable for small projects.

8.4 Comparison of Models

Factor	Waterfall	Agile	Spiral
Approach	Linear, one phase at a time	Short repeating sprints	Repeating loops with risk analysis
Handling change	Difficult	Very easy	Moderate
Customer involvement	Low	High and continuous	Medium
Risk handling	Weak	Good through feedback	Excellent
Cost	Low	Moderate	High
Best for	Small projects with fixed needs	Projects with changing needs	Large and risky projects

8.5 Model Selected for the Case

For the OLMS project the team selects the Agile (Scrum) model. The library staff are not sure about every feature at the start, and they want to try the search and issue modules early and suggest changes. Agile allows this. The project is also small, so the cost and effort of a Spiral approach are not justified, while Waterfall would make later changes expensive. Formal documents such as the SRS and design diagrams are still prepared, so the project keeps the discipline of the SDLC.

9 Applying the SDLC to the Case

This section follows the six SDLC phases one by one and shows what was done for the Online Library Management System.

9.1 Phase 1: Planning

The team first checked whether the project is worth doing through a feasibility study, and then prepared a schedule.

Feasibility	Finding	Result
Technical	Web technologies and a standard database are available; team has the skills.	Feasible
Economic	One-time development cost is low compared with the time saved by staff every day.	Feasible
Operational	Staff and students are comfortable with computers and ready to use the system.	Feasible

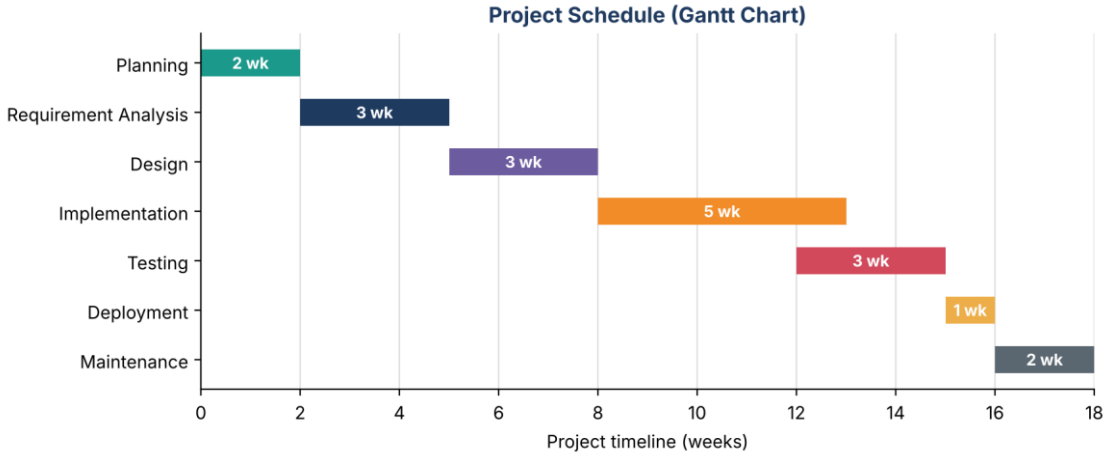


Fig. 5: Project schedule (Gantt chart), 18 weeks in total

9.2 Phase 2: Requirement Analysis

Requirements were collected through interviews with the librarian and a survey of students. They were written down in a Software Requirement Specification (SRS).

ID	Functional requirement	Priority
FR-1	A member can register and log in with a user ID and password.	High
FR-2	A user can search books by title, author or subject.	High
FR-3	The librarian can issue and return books and renew a loan.	High
FR-4	The system calculates the fine automatically for late returns.	Medium
FR-5	The librarian can add, edit or delete book records.	High
FR-6	The admin can generate reports and manage user accounts.	Medium

Non-functional requirements: search results within 3 seconds; support at least 100 users at the same time; passwords stored in encrypted form; simple screens that need no training; daily automatic backup of data.

9.3 Phase 3: System Design

In the design phase the requirements are converted into diagrams that developers can follow. Three kinds of diagrams are used here: a process flowchart, a use case diagram and Data Flow Diagrams.

A. Flowchart of the development process. A flowchart uses standard shapes to show steps in order. Rounded boxes mark the start and end, rectangles are activities and the diamond is a decision.

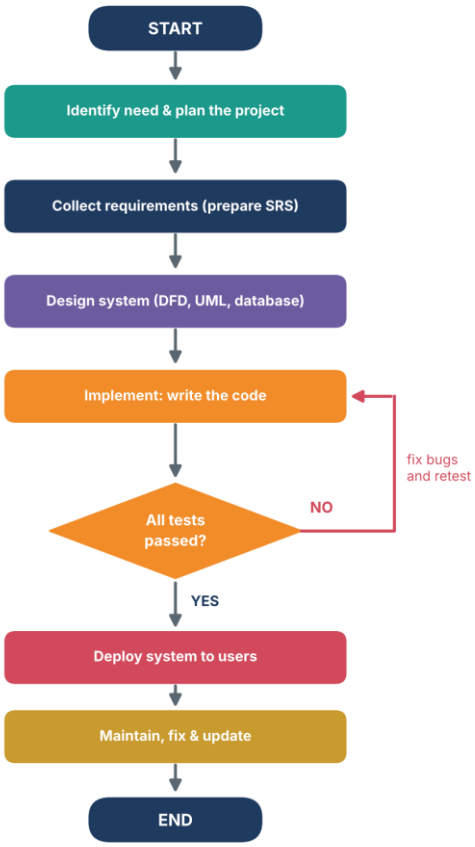


Fig. 6: Flowchart of the development process

How to read the flowchart

1. The process starts when the library identifies a need, and a plan is made.
2. Requirements are collected and written as an SRS.
3. The system is designed with DFDs, UML and a database.
4. Developers write the code.
5. The software is tested. If any test fails (NO), the bugs are fixed and the code is tested again.
6. When all tests pass (YES), the system is deployed to users.
7. After release, the team keeps maintaining it by fixing errors and adding updates.

B. Use case diagram. A use case diagram shows who uses the system (actors) and what they can do (use cases). In the OLMS, the member searches books and can request issue and return, while the librarian handles issue, return and record management.

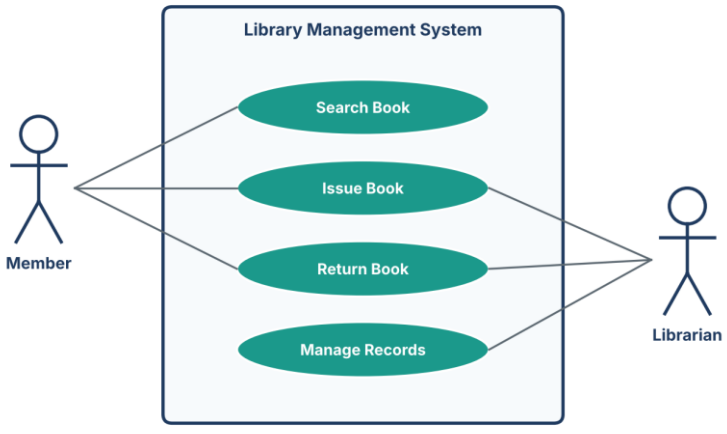


Fig. 7: Use case diagram of the Online Library Management System

C. Data Flow Diagrams (DFDs). A DFD shows how data moves through a system: where it comes from, what processes change it, where it is stored and where it goes. It does not show program code or the order of steps. A DFD is drawn in levels, starting with a simple overall picture and then adding detail.

Symbol	Name	Meaning
Rectangle	External entity	A person or system outside our system that sends or receives data (e.g., Member, Librarian).
Circle	Process	An activity that changes incoming data into outgoing data (e.g., Issue / Return).
Open-ended parallel lines	Data store	A place where data is kept (e.g., Books, Members).
Arrow	Data flow	The direction in which data moves, labelled with the data name.

Level 0 DFD (context diagram). The whole OLMS is shown as one single process. The diagram only shows the external users and the main data they exchange with the system.

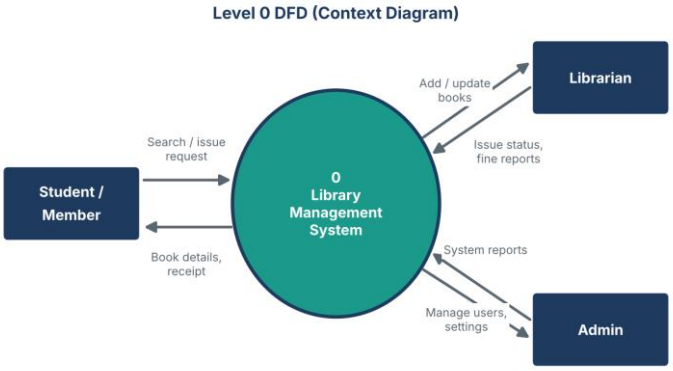


Fig. 8: Level 0 DFD (context diagram)

Level 1 DFD. The single process is opened up into four major processes: Register / Login, Search Books, Issue / Return, and Generate Fine & Reports. The three data stores hold member details, book records and issue records. The Member logs in, searches the catalogue and requests an issue or return. The Librarian receives fine reports and updates book records.

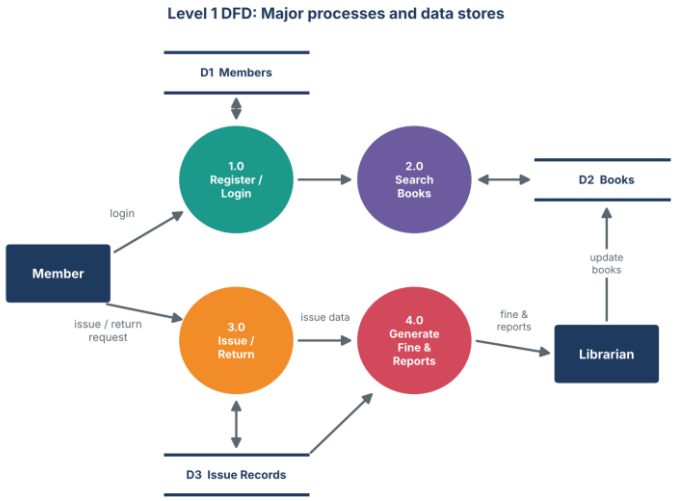


Fig. 9: Level 1 DFD of the Online Library Management System

9.4 Phase 4: Implementation

In this phase the developers converted the design into working code, one sprint at a time. The first sprint delivered login and book search, the second delivered issue and return, and the third delivered fines and reports. The source code was kept in a Git repository so that every change could be tracked.

Layer	Technology used
Front end (screens)	HTML, CSS and JavaScript
Back end (logic)	Python (Flask) or Java, any language the team knows well
Database	MySQL with tables for Members, Books and Issue Records
Version control	Git and GitHub

9.5 Phase 5: Testing

Testing was carried out at three levels. Unit testing checks single functions, integration testing checks that modules work together, and system testing checks the complete product against the SRS. A few sample test cases are given below.

Test ID	Test case	Expected result	Status
TC-01	Login with a valid user ID and password	User enters the dashboard	Pass
TC-02	Login with a wrong password	Error message is displayed	Pass
TC-03	Search for an existing book title	Matching book list is shown	Pass
TC-04	Issue a book that is already issued	System refuses and shows status	Pass
TC-05	Return a book 5 days late	Fine is calculated correctly	Pass
TC-06	Add a book with an empty title	Validation error is shown	Fixed, then Pass

9.6 Phase 6: Deployment and Maintenance

After the final tests, the system was installed on the college server. Staff received a short training session and the old records were entered into the new database. Once the system is live, maintenance continues in four forms:

- **Corrective:** fixing bugs found by users.
- **Adaptive:** changing the system when the environment changes (for example a new browser version).
- **Perfective:** adding improvements and new features, such as email reminders.
- **Preventive:** cleaning up code and taking backups to avoid future problems.

10 Findings and Analysis

Comparing the manual system with the new system gave the following improvements (expected values from the case scenario):

Activity	Manual system	OLMS
Searching a book	5 to 10 minutes	Under 3 seconds
Issuing a book	About 3 minutes	About 20 seconds
Fine calculation	By hand, error-prone	Automatic and accurate
Monthly report	Half a day of effort	Generated in one click
Data safety	Paper can be lost	Daily backup

11 Challenges and Lessons Learned

- **Changing requirements:** the librarian asked for renewals late in the project. Agile sprints made it easy to add them.
- **Communication:** regular stand-up meetings avoided misunderstandings between the developers and the tester.
- **Lesson:** time spent on requirements and design saved far more time during coding and testing.

12 Conclusion

This case study showed that the SDLC is not just theory but a practical roadmap that guides a project from idea to a working product. By choosing the Agile model and carefully completing the planning, requirement, design, implementation, testing and maintenance phases, the Online Library Management System was delivered with fewer errors and with greater user satisfaction. The diagrams such as the flowchart and DFDs made the system easy to understand for both technical and non-technical people. The study concludes that choosing the right model and following the life cycle with discipline are the keys to successful software.

References

[1] Pressman, R. S., and Maxim, B. R., Software Engineering: A Practitioner's Approach, McGraw-Hill Education.

[2] Sommerville, I., Software Engineering, Pearson Education.

[3] Pankaj Jalote, An Integrated Approach to Software Engineering, Narosa Publishing House.

[4] Schwaber, K., and Sutherland, J., The Scrum Guide.